

PowerShell

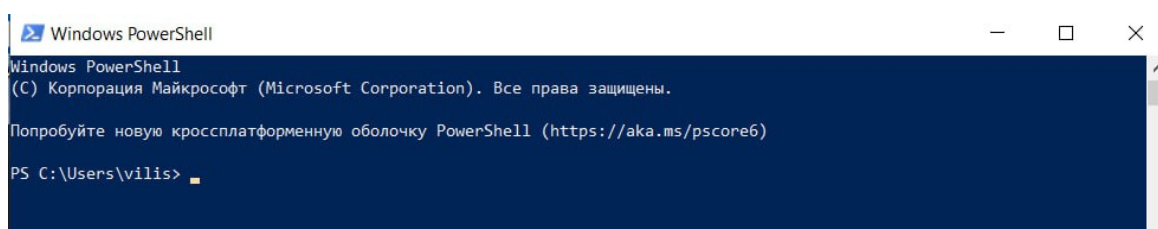
Kyrylo Pryiomyshev
Windows Administration
October 2025

What is PowerShell?

PowerShell is a task automation and configuration management framework developed by Microsoft. Unlike the old Command Prompt, which was limited to text-based commands, PowerShell is built on top of the .NET framework and works with objects instead of plain text. This means that when a command is executed, the output is not just a string, but a structured object that can be filtered, sorted, and passed to other commands.

PowerShell is used by system administrators and IT professionals to automate repetitive tasks, configure systems at scale, and troubleshoot complex environments. It is equally useful for developers and DevOps engineers, who often need to script deployments, manage cloud resources, or process log data efficiently.

Based PowerShell terminal



Basics of PowerShell

PowerShell fundamentals begin with understanding the environment and the commands (called cmdlets) that it provides. Cmdlets follow a consistent **Verb-Noun** pattern, such as '*Get-Process*' or '*Set-Content*', which makes them easier to learn and remember.

'*\$PSVersionTable*' displays the version of PowerShell and its runtime details, while '*Get-Host*' provides information about the host program itself. These are particularly important when running scripts across different machines, as some features are only available in newer versions.

```
PS C:\Users\vilis> $PSVersionTable

Name                           Value
----                           -
PSVersion                      5.1.19041.610
PSEdition                     Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.19041.610
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1

PS C:\Users\vilis> Get-Host

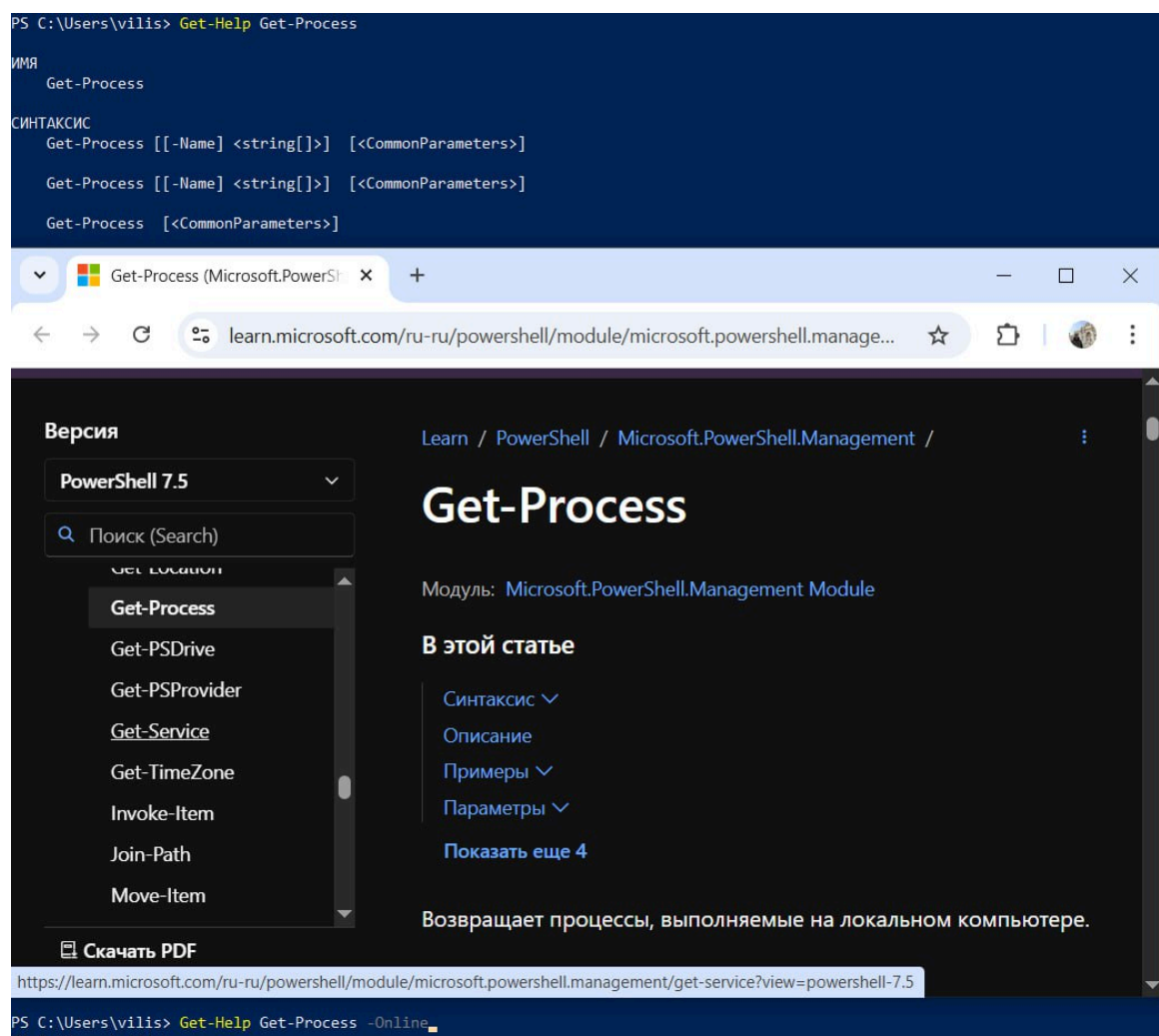
Name           : ConsoleHost
Version        : 5.1.19041.610
InstanceId     : a33e3d98-96f4-4abc-a18d-2e6b270a8cbc
UI             : System.Management.Automation.Internal.Host.InternalHostUserInterface
CurrentCulture : ru-RU
CurrentUICulture : ru-RU
PrivateData    : Microsoft.PowerShell.ConsoleHost+ConsoleColorProxy
DebuggerEnabled : True
IsRunspacePushed : False
Runspace       : System.Management.Automation.Runspace.LocalRunspace
```

The command '*Get-Command | Measure-Object*' counts the available commands and shows how extensive the environment is.

```
PS C:\Users\vilis> Get-Command | Measure-Object

Count      : 1491
Average    :
Sum        :
Maximum    :
Minimum    :
Property   :
```

The *'Get-Help'* cmdlet is the built-in documentation system. It can display full descriptions, syntax, and examples of any cmdlet. For beginners, the *'-Examples'* parameter is especially useful because it provides practical usage scenarios. If more details are needed, the *'-Online'* parameter opens the official Microsoft Docs page in a browser.



Script execution policy

By default, Windows PowerShell prevents the execution of scripts to protect the system from untrusted code. The command ‘*Get-ExecutionPolicy -List*’ shows the current restrictions. There are several other execution policy levels available. To establish an execution policy, the command ‘*Set-ExecutionPolicy*’ must be used. It is possible to specify not only the policy level, but also additional parameters. For example, the **-Scope** flag determines whether the policy applies only to the current PowerShell process, to the current user, or to all users on the machine.

Execution Policy Levels	Level's Behavior
RemoteSigned	Allows local scripts to run, while still blocking unsigned scripts from the internet.
Restricted	Is the most secure mode, where no scripts are allowed to run at all.
AllSigned	Requires all scripts, even those created locally, to be digitally signed by a trusted publisher.
Unrestricted	Allows any script to run, but scripts downloaded from the internet trigger a warning before execution.
Bypass	Removes all restrictions and warning entirely, which is useful in automated environments but poses the greatest security risk.
Undefined	Means that no explicit policy is set in the given scope, and PowerShell will fall back to higher-level configuration.

```
PS C:\Users\vilis> Get-ExecutionPolicy -List
Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser Undefined
LocalMachine Restricted

PS C:\Users\vilis> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
Изменение политики выполнения
Политика выполнения защищает компьютер от ненадежных сценариев. Изменение политики выполнения может поставить под угрозу безопасность системы, ка
по адресу https://go.microsoft.com/fwlink/?LinkID=135170 . Вы хотите изменить политику выполнения?
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S] Приостановить - S [?] Справка (значением по умолчанию является "N"): Y
PS C:\Users\vilis> Get-ExecutionPolicy -List
Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser RemoteSigned
LocalMachine Restricted
```

File System Operations

Exploring the system, Navigation and Creating files

PowerShell provides administrators and users with immediate visibility into what is happening on a system. The command '*Get-Process*' enumerates all currently running processes, while '*Get-Service*' displays the services installed on the machine along with their status.

PowerShell also facilitates navigation within the file system. The command '*Get-Location*' shows the current working directory, and Set-Location can be used to change it. Combined with '*Get-ChildItem*', which lists the files and subdirectories contained in a given path, these commands form the foundation for virtually all file system operations.

PowerShell also includes commands for managing items in the file system. With '*New-Item*', both files and directories can be created by specifying the desired path, name, and type. Otherwise, '*Remove-Item*' deletes files or directories when they are no longer needed. Deployment scripts often rely on such commands to ensure that the necessary directory structure exists or that obsolete files are removed before installing updated components.

```
PS C:\Users\vilis> Get-Service
```

Status	Name	DisplayName
Stopped	AarSvc_1bb1708	Agent Activation Runtime_1bb1708
Running	AdobeARMservice	Adobe Acrobat Update Service
Stopped	AJRouter	Служба маршрутизатора AllJoyn
Stopped	ALG	Служба шлюза уровня приложения

```
PS C:\Users\vilis> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
428	30	36492	4888	0,45	6096	5	AcerRegistrationBackGroundTask
535	29	25396	2884	1,00	8772	5	ApplicationFrameHost
155	9	1904	1384	0,03	11044	5	AppVShNotify


```
PS C:\Users\vilis> New-Item -Path . -Name "demo.txt" -ItemType File

Karanor: C:\Users\vilis

Mode                LastWriteTime         Length Name
----                -
-a-----         01.10.2025         14:08             0 demo.txt

PS C:\Users\vilis> New-Item -Path . -Name "Projects"-ItemType Directory

Karanor: C:\Users\vilis

Mode                LastWriteTime         Length Name
----                -
d-----         01.10.2025         14:09             Projects

PS C:\Users\vilis> Remove-Item .\demo.txt
```

```
PS C:\Users\vilis> Get-Location

Path
----
C:\Users\vilis
```

```
PS C:\Users\vilis> Remove-Item .\demo.txt
PS C:\Users\vilis> Get-ChildItem

Karanor: C:\Users\vilis

Mode                LastWriteTime         Length Name
----                -
d-----         11.02.2025         16:47         .arduinoIDE
d-----         02.09.2025         15:12         .cache
d-----         13.04.2023         14:06         .dotnet
d-----         18.11.2024         21:04         .idlerc
d-----         25.08.2025         22:08         .junie
d-----         23.11.2024         21:14         .librarymanager
d-----         08.11.2024         17:05         .matplotlib
d-----         10.05.2023         19:41         .ms-ad
d-----         05.09.2025         17:15         .ngrok2
d-----         16.02.2023         20:39         .Origin
d-----         16.02.2023         20:39         .QtWebEngineProcess
d-----         17.02.2023         13:58         .templateengine
d-r---         17.02.2023             8:33         3D Objects
d-r---         16.04.2025         12:37         Cisco Packet Tracer 8.2.2
d-r---         17.02.2023             8:33         Contacts
d-----         17.02.2023         13:28         Documents
d-r---         05.09.2025         11:12         Downloads
d-r---         17.02.2023             8:33         Favorites
d-r---         17.02.2023             8:33         Links
d-r---         17.02.2023             8:33         Music
d-----         27.08.2025         17:18         npm-cache
dar--l         14.04.2025         16:18         OneDrive
d-----         01.10.2025         14:09         Projects
d-----         27.08.2025         10:43         PycharmProjects
d-r---         10.07.2023         18:57         Saved Games
d-r---         17.02.2023             8:34         Searches
d-----         10.04.2023         14:07         source
d-r---         14.02.2025             9:46         Videos
d-----         02.09.2025         14:56         WebstormProjects
d-----         27.08.2025         10:57         work_bot
d-----         17.02.2023         13:56         Проекты в c#
-a-----         12.02.2024         17:16         7024 -1.14-windows.xml
-a-----         19.11.2024         14:30             9 .bash_history
-a-----         19.11.2024         13:14             57 .gitconfig
-a-----         19.11.2024         15:56             10 .gitignore
-a-----         27.08.2025         19:42             20 .lessht
-a-----         16.04.2025         12:37            176 .packettracer
-a-----         05.09.2025         14:57         2491 .viminfo
```

Reading and Writing files

PowerShell provides built-in cmdlets for creating and manipulating the contents of text files without the need to open a graphical editor. For example:

```
Set-Content -Path .\example.txt -Value "text"
```

```
Add-Content -Path .\example.txt -Value "next line text"
```

```
Get-Content -Path .\example.txt
```

The '*Set-Content*' cmdlet writes data to a file, creating it if it does not already exist and overwriting its contents if it does. '*Add-Content*' appends new data to the end of an existing file, preserving what was already stored. To retrieve the contents of a file, the '*Get-Content*' cmdlet is used, which outputs each line as a string that can be further processed or piped into other commands.

```
PS C:\Users\vilis> Set-Content -Path .\example.txt -Value "Tere, PowerShell!"
PS C:\Users\vilis> Add-Content -Path .\example.txt -Value "Vabanda, PowerShell, aga bash parem ja zsh parim"
PS C:\Users\vilis> Get-Content -Path .\example.txt
Tere, PowerShell!
Vabanda, PowerShell, aga bash parem ja zsh parim
```

Copying and moving files

PowerShell also provides commands for duplicating or relocating files within the file system. '*The Copy-Item*' cmdlet creates a duplicate of a file or directory at the specified destination, while '*Move-Item*' transfers the original object to a new location. Both cmdlets support additional parameters, such as **-Recurse** for operating on entire directory structures, or **-Force** to overwrite existing files without confirmation.

These operations are particularly useful in administrative scripts. For instance, automated backup routines can rely on '*Copy-Item*' to safeguard critical files at regular intervals, whereas '*Move-Item*' can be employed to archive or reorganize data once it is no longer actively used.

```
PS C:\Users\vilis> Copy-Item .\example.txt c:\Users\vilis\Projects
PS C:\Users\vilis> Move-Item .\example.txt C:\Users\vilis\OneDrive
```


User Management

Viewing local users

PowerShell provides a powerful interface for administering local user accounts and groups. These commands enable administrators to create, modify, and remove accounts directly from the command line, thereby streamlining common management tasks and making them easily automatable.

The '*Get-LocalUser*' cmdlet retrieves all local accounts on a system. Combined with '*Select-Object*', administrators can quickly filter relevant properties, such as whether an account is enabled and the date of its last logon. This functionality allows for efficient auditing and monitoring of account usage.

```
PS C:\Users\vilis> Get-LocalUser

Name                Enabled Description
-----
DefaultAccount      False  Учетная запись пользователя, управляемая системой.
vilis                True
WDAGUtilityAccount  False  Учетная запись пользователя, которая управляется и используется системой для сценариев Application Guard в Защитнике Windows.
Администратор        False  Встроенная учетная запись администратора компьютера/домена
Гость                False  Встроенная учетная запись для доступа гостей к компьютеру или домену

PS C:\Users\vilis> Get-LocalUser | Select-Object Name, Enabled

Name                Enabled
-----
DefaultAccount      False
vilis                True
WDAGUtilityAccount  False
Администратор        False
Гость                False
```

Creating a new user

With '*New-LocalUser*', accounts can be created directly from the command line, providing full control over attributes such as name, password, and description. Compared to using the graphical user interface, this method is faster, less error-prone, and more scalable when multiple accounts must be provisioned.

```
PS C:\Windows\system32> $Password = Read-Host "Enter password" -AsSecureString
Enter password: *****
PS C:\Windows\system32> New-LocalUser -Name "example_user" -Password $Password -FullName "Tere Tulemast" -Description "For presentation"

Name                Enabled Description
-----
example_user        True    For presentation

PS C:\Windows\system32> Get-LocalUser | Select-Object Name

Name
-----
DefaultAccount
example_user
vilis
WDAGUtilityAccount
Администратор
Гость

PS C:\Windows\system32>
```

Managing accounts

Local accounts can be temporarily disabled, re-enabled when needed, or permanently removed.

```
PS C:\Windows\system32> Disable-LocalUser -Name "example_user"
PS C:\Windows\system32> Get-LocalUser | Select-Object Name, Enabled
```

Name	Enabled
-----	-----
DefaultAccount	False
<u>example_user</u>	False
vilis	True
WDAGUtilityAccount	False
Администратор	False
Гость	False


```
PS C:\Windows\system32> Enable-LocalUser -Name "example_user"
PS C:\Windows\system32> Get-LocalUser | Select-Object Name, Enabled
```

Name	Enabled
-----	-----
DefaultAccount	False
<u>example_user</u>	True
vilis	True
WDAGUtilityAccount	False
Администратор	False
Гость	False


```
PS C:\Windows\system32> Remove-LocalUser -Name "example_user"
PS C:\Windows\system32> Get-LocalUser | Select-Object Name
```

Name

DefaultAccount
vilis
WDAGUtilityAccount
Администратор
Гость

 **Account is deleted**

Working with groups

Groups serve as the primary mechanism for assigning permissions. The ‘*Get-LocalGroup*’ cmdlet lists all groups on the system, while ‘*Get-LocalGroupMember*’ allows administrators to see which users are part of a given group.

Membership in groups can be granted or revoked programmatically. This enables fine-grained control of privileges and ensures that users have only the access rights they require. For example:

Add-LocalGroupMember -Group "Hyper-V Administrators" -Member "vilis"

Get-LocalGroupMember -Group "Hyper-V Administrators"

Remove-LocalGroupMember -Group "Hyper-V Administrators" -Member "vilis"

```
PS C:\Windows\system32> Get-LocalGroup
Name
----
Hyper-V Administrators
IIS_IUSRS
Администраторы
Пользователи
Пользователи DCIM
Пользователи журналов производительности
Пользователи системного монитора
Пользователи удаленного управления
Управляемая системой группа учетных записей
Читатели журнала событий
Description
-----
Members of this group have complete and unrestricted access to all features of Hyper-V.
Встроенная группа, которую использует служба IIS.
Администраторы имеют полные, ничем не ограниченные права доступа к компьютеру или домену
Члены этой группы могут изменять параметры на уровне системы.
Гости по умолчанию имеют те же права, что и пользователи, за исключением учетной записи "Гость", еще более ограниченной в правах.
Пользователи не имеют прав на изменение параметров системы и могут запускать большинство приложений
Члены этой группы могут запускать, активизировать и использовать объекты DCIM на этом компьютере.
Члены этой группы могут планировать издание журнала счетчиков производительности, включать поставщиков трассировки и собирать трассировки событий как локально, так и через удаленный доступ ...
Члены данной группы имеют как местный, так и удаленный доступ к счетчику производительности
Члены этой группы могут получать доступ к ресурсам инструментария WMI по протоколам управления (таким как WS-Management в службе удаленного управления Windows). Это касается только простран...
Члены этой группы могут читать журналы событий с локального компьютера

PS C:\Windows\system32> Add-LocalGroupMember -Group "Hyper-V Administrators" -Member "vilis"
PS C:\Windows\system32> Get-LocalGroupMember -Group "Hyper-V Administrators"

ObjectClass Name PrincipalSource
-----
Пользователь LAPT0P-4ABP1F8B\vilis MicrosoftAccount

PS C:\Windows\system32> Remove-LocalGroupMember -Group "Hyper-V Administrators" -Member "vilis"
PS C:\Windows\system32> Get-LocalGroupMember -Group "Hyper-V Administrators"
PS C:\Windows\system32>
```

Networking in PowerShell

PowerShell consolidates a wide range of networking capabilities into a consistent set of cmdlets, replacing many traditional command-line utilities such as **ipconfig**, **ping**, and **nslookup**. These cmdlets provide structured, scriptable output and support parameters that allow fine-grained control, making them particularly effective for both diagnostics and automation.

Viewing Configuration

The '*Get-NetIPAddress*' cmdlet lists all IP addresses assigned to the system's network interfaces. '*Get-NetIPConfiguration*' extends this by including details such as default gateways, DNS servers, and adapter properties. The structured output allows administrators to filter and export relevant information for auditing or troubleshooting.

```
PS C:\Windows\system32> Get-NetIPAddress

IPAddress      : fe80::df3:7b07:899b:b8c9%16
InterfaceIndex : 16
InterfaceAlias : Подключение по локальной сети* 2
AddressFamily  : IPv6
Type           : Unicast
PrefixLength   : 64
PrefixOrigin   : WellKnown
SuffixOrigin    : Link
AddressState    : Deprecated
ValidLifetime  : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime : Infinite ([TimeSpan]::MaxValue)
SkipAsSource    : False
PolicyStore     : ActiveStore

IPAddress      : fe80::4cc7:41ea:30a8:1a8b%10
InterfaceIndex : 10
InterfaceAlias : Подключение по локальной сети* 1
AddressFamily  : IPv6
Type           : Unicast
PrefixLength   : 64
PrefixOrigin   : WellKnown
SuffixOrigin    : Link
AddressState    : Deprecated
ValidLifetime  : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime : Infinite ([TimeSpan]::MaxValue)
SkipAsSource    : False
PolicyStore     : ActiveStore

IPAddress      : fe80::b00c:c2b3:1c86:dbb4%7
InterfaceIndex : 7
InterfaceAlias : Сетевое подключение Bluetooth
AddressFamily  : IPv6
Type           : Unicast
PrefixLength   : 64
PrefixOrigin   : WellKnown
SuffixOrigin    : Link
```

```
PS C:\Windows\system32> Get-NetIPConfiguration

InterfaceAlias : Беспроводная сеть
InterfaceIndex : 9
InterfaceDescription : Intel(R) Wireless-AC 9462
NetProfile.Name : campusnet 2
IPv4Address     : 172.26.20.40
IPv6DefaultGateway :
IPv4DefaultGateway : 172.26.20.1
DNSServer       : 195.80.96.222
                195.80.103.200

InterfaceAlias : Сетевое подключение Bluetooth
InterfaceIndex : 7
InterfaceDescription : Bluetooth Device (Personal Area Network)
NetAdapter.Status : Disconnected

InterfaceAlias : Ethernet
InterfaceIndex : 3
InterfaceDescription : Realtek PCIe GbE Family Controller
NetAdapter.Status : Disconnected
```

Testing connectivity

‘*Test-Connection*’ is a replacement for **ping**, providing latency values and packet statistics. The parameter **-Count 4** explicitly defines the number of ICMP echo requests sent (4 in this case), ensuring consistency in test results.

‘*Test-NetConnection*’ verifies whether a host is reachable and optionally whether a specific port is open. The flag **-Port 443** targets the HTTPS service, allowing administrators to confirm not only that the host responds but also that the intended service is accessible.

```
PS C:\Windows\system32> Test-Connection google.com -Count 4

Source      Destination    IPv4Address    IPv6Address    Bytes    Time(ms)
-----
LAPTOP-4AB... google.com      172.217.21.174    32         18
LAPTOP-4AB... google.com      172.217.21.174    32         19
LAPTOP-4AB... google.com      172.217.21.174    32         20
LAPTOP-4AB... google.com      172.217.21.174    32         18

PS C:\Windows\system32> Test-NetConnection google.com -Port 443

ComputerName : google.com
RemoteAddress : 172.217.21.174
RemotePort    : 443
InterfaceAlias : Беспроводная сеть
SourceAddress  : 172.26.20.40
TcpTestSucceeded : True
```

Managing adapters

The ‘*Get-NetAdapter*’ cmdlet retrieves information about available network interfaces, including their status and properties. Interfaces can be disabled or re-enabled directly from the console using ‘*Disable-NetAdapter*’ and ‘*Enable-NetAdapter*’. The parameter **-Confirm:\$false** bypasses the usual confirmation prompt, which is essential in automated scripts where user interaction is not possible.

```
PS C:\Windows\system32> Get-NetAdapter

Name                InterfaceDescription      ifIndex Status    MacAddress      LinkSpeed
-----
Беспроводная сеть   Intel(R) Wireless-AC 9462    9 Up        0C-9A-3C-0C-13-36 150 Mbps
Сетевое подключение Bluetooth Device (Personal Area Netw... 7 Disconnected 0C-9A-3C-0C-13-3A 3 Mbps
Ethernet             Realtek PCIe GbE Family Controller 3 Disconnected 08-8F-C3-10-6D-6D 0 bps

PS C:\Windows\system32> Disable-NetAdapter -Name "Ethernet" -Confirm:$false
PS C:\Windows\system32> Get-NetAdapter

Name                InterfaceDescription      ifIndex Status    MacAddress      LinkSpeed
-----
Беспроводная сеть   Intel(R) Wireless-AC 9462    9 Up        0C-9A-3C-0C-13-36 150 Mbps
Сетевое подключение Bluetooth Device (Personal Area Netw... 7 Disconnected 0C-9A-3C-0C-13-3A 3 Mbps
Ethernet             Realtek PCIe GbE Family Controller 3 Disabled    08-8F-C3-10-6D-6D 0 bps

PS C:\Windows\system32> Enable-NetAdapter -Name "Ethernet"
PS C:\Windows\system32> Get-NetAdapter

Name                InterfaceDescription      ifIndex Status    MacAddress      LinkSpeed
-----
Беспроводная сеть   Intel(R) Wireless-AC 9462    9 Up        0C-9A-3C-0C-13-36 150 Mbps
Сетевое подключение Bluetooth Device (Personal Area Netw... 7 Disconnected 0C-9A-3C-0C-13-3A 3 Mbps
Ethernet             Realtek PCIe GbE Family Controller 3 Disconnected 08-8F-C3-10-6D-6D 0 bps
```


DNS

‘*Resolve-DnsName*’ provides a structured alternative to *nslookup*, displaying DNS records in a format suitable for further processing. ‘*Get-DnsClientServerAddress*’ retrieves the DNS server settings for each adapter, enabling administrators to verify resolution paths and detect potential misconfigurations.

```
PS C:\Windows\system32> Resolve-DnsName google.com

Name                                     Type  TTL  Section  IPAddress
----
google.com                             AAAA   246  Answer   2a00:1450:400f:80a::200e
google.com                             A      199  Answer   172.217.21.174

PS C:\Windows\system32> Get-DnsClientServerAddress_

InterfaceAlias      Interface Index Address ServerAddresses
-----
Ethernet            3 IPv4   {192.168.0.1}
Ethernet            3 IPv6   {}
Подключение по локальной...1  10 IPv4   {}
Подключение по локальной...1  10 IPv6   {fec0:0:0:ffff::1, fec0:0:0:ffff::2, fec0:0:0:ffff::3}
Подключение по локальной...2  16 IPv4   {}
Подключение по локальной...2  16 IPv6   {fec0:0:0:ffff::1, fec0:0:0:ffff::2, fec0:0:0:ffff::3}
Беспроводная сеть  9 IPv4   {195.80.96.222, 195.80.103.200}
Беспроводная сеть  9 IPv6   {}
Сетевое подключение Bluetooth... 7 IPv4   {}
Сетевое подключение Bluetooth... 7 IPv6   {fec0:0:0:ffff::1, fec0:0:0:ffff::2, fec0:0:0:ffff::3}
Loopback Pseudo-Interface 1  1 IPv4   {}
Loopback Pseudo-Interface 1  1 IPv6   {fec0:0:0:ffff::1, fec0:0:0:ffff::2, fec0:0:0:ffff::3}
```

Firewall management

The Windows Firewall can be managed from PowerShell. ‘*Get-NetFirewallProfile*’ reveals the status of firewall profiles, while ‘*Get-NetFirewallRule*’ filters rules by group for auditing. With ‘*Set-NetFirewallProfile*’, administrators can enable or disable firewall profiles across multiple scopes (Domain, Public, and Private). The ability to toggle settings programmatically is crucial for testing connectivity scenarios and managing system security at scale.

```
PS C:\Windows\system32> Get-NetFirewallProfile

Name           : Domain
Enabled        : True
DefaultInboundAction : NotConfigured
DefaultOutboundAction : NotConfigured
AllowInboundRules : NotConfigured
AllowLocalFirewallRules : NotConfigured
AllowLocalIPsecRules : NotConfigured
AllowUserApps  : NotConfigured
AllowUserPorts : NotConfigured
AllowUnicastResponseToMulticast : NotConfigured
NotifyOnListen : True
EnableStealthModeForIPsec : NotConfigured
LogFileNames   : %systemroot%\system32\LogFiles\Firewall\pfirewall.log
LogMaxSizeKilobytes : 4096
LogAllowed     : NotConfigured
LogBlocked     : NotConfigured
LogIgnored     : NotConfigured
DisabledInterfaceAliases : {NotConfigured}

Name           : Private
```


Automation with PowerShell

Automation represents the core strength of PowerShell. By leveraging variables, loops, conditional logic, functions, and scripts, administrators can perform complex and repetitive tasks with minimal manual intervention. This reduces the potential for human error, improves efficiency, and allows for scalable system management.

Variables and data types

Variables in PowerShell can hold values of different data types, including strings, integers, Booleans, and arrays. In the example below, **\$Name** stores a *string*, **\$Age** an *integer*, **\$IsAdmin** a *Boolean value*, and **\$List** an *array of integers*. Arrays such as **\$List** can later be iterated over in loops, while simple variables like **\$Name** and **\$Age** can be combined into conditions or output statements. This makes scripts reusable and adaptable to different contexts.

```
PS C:\Windows\system32> $Name = "Messmer"
PS C:\Windows\system32> $Age = 305
PS C:\Windows\system32> $IsAdmin = $true
PS C:\Windows\system32> $List = 1, 2, 3, 4, 5
```

Loops and conditions

Loops and conditional statements enable the automation of repetitive tasks and decision-making. The *for* loop iterates a fixed number of times, while *foreach* processes each item in a collection, such as files in a directory. Conditional logic (*if* statements) allows administrators to execute actions only when certain criteria are met. This structure is fundamental for tasks like processing log files, monitoring system states, or applying policies selectively.

In the example below the *for* loop initializes a counter **\$i** at 1 and runs as long as **\$i** is less than or equal to 5. After each iteration, **\$i** is incremented by 1. The result is five outputs, labeled 'Iteration 1' through 'Iteration 5'; in the *foreach* example, the **\$files** variable holds all text files in the current directory. The loop processes each file in turn, outputting its name. This is a common pattern for batch file processing.

The *if* statement checks whether **\$Age** is greater than or equal to 18. If the condition evaluates to true, it outputs that the user is an adult.

```
PS C:\Windows\system32> for ($i=1; $i -le 5; $i++) { Write-Output "Iteration $i" }
>> $files = Get-ChildItem *.txt
>> foreach ($file in $files) {
>>     Write-Output "Processing $($file.Name)"
>> }
>> if ($Age -ge 18) { Write-Output "$Name is an adult" }
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Processing ThirdPartyNoticesBySHS.txt
Processing WindowsCodecsRaw.txt
Messmer is an adult
```

Functions

Functions in PowerShell help avoid repetition by grouping reusable logic, and they can accept parameters to make them more versatile.

In the example below the function ‘*Get-Square*’ encapsulates logic that multiplies a number by itself and returns the result. When called with the argument 5, the output is 25.

```
PS C:\Windows\system32> function Get-Square($number) {
>>     return $number * $number
>> }
>> Get-Square 5
25
```

Creating and running scripts

Scripts are saved with the **.ps1** extension and executed directly in PowerShell. In this example below, the variable **\$computers** contains a list of three hostnames: PC1, PC2, and PC3. The *foreach loop* iterates over each element in this array. For every host, two actions are performed: ‘*Write-Output*’ prints a message indicating which computer is being tested; ‘*Test-Connection*’ sends a single ICMP echo request (-Count 1) to the host. The **-Quiet** flag ensures that the result is returned as a Boolean value: True if the host replies, False if it is unreachable. In the example output, each host returns False, meaning none of the specified hostnames responded to the ping request.

These scripts are usually used in administration to test the availability of multiple systems. By substituting real hostnames or IP addresses, the same structure can serve as a monitoring or diagnostic tool.

```
PS C:\Windows\system32> # demo.ps1
>> $computers = "PC1","PC2","PC3"
>> foreach ($c in $computers) {
>>     Write-Output "Pinging $c..."
>>     Test-Connection $c -Count 1 -Quiet
>> }
Pinging PC1...
False
Pinging PC2...
False
Pinging PC3...
False
```

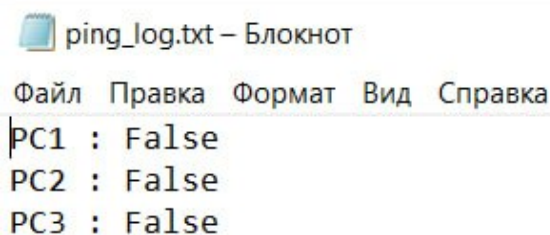
Scheduling tasks

Scheduling allows scripts to run automatically without user interaction. In this example below, *'Register-ScheduledTask'* creates a new scheduled task that will execute PowerShell and run the script **demo.ps1**. The trigger specifies that it should start once, one minute from the current time *'((Get-Date).AddMinutes(1))'*. By combining actions and triggers, administrators can schedule recurring tasks such as log collection, backups, or system checks, ensuring consistency and reducing the need for manual execution.

It is important to note that in this scenario the script runs in the background, and its output will not be displayed in the PowerShell terminal. To track execution, the script can be modified to write results to a log file.

For example:

```
PS C:\Windows\system32> Register-ScheduledTask -Action (New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument ". C:\PSDemo\PSAutomation\demo.ps1")
PS C:\Windows\system32> $computers = "PC1","PC2","PC3"
>> foreach ($c in $computers) {
>>     $result = Test-Connection $c -Count 1 -Quiet
>>     "$c : $result" | Out-File "C:\PSDemo\ping_log.txt" -Append
>> }
```



ping_log.txt – Блокнот

Файл Правка Формат Вид Справка

PC1 : False
PC2 : False
PC3 : False

The script can also be modified to record the date and time of execution. Such flexible settings allow you to automate many routine processes and simplify your work to checking logs.

Summary

PowerShell has evolved into far more than a simple command-line tool: it represents a comprehensive automation and administration environment that unifies system management tasks under a consistent syntax. Its strength lies in combining access to operating system features with the ability to script and automate repetitive actions. Through a wide range of cmdlets, administrators can interact with the file system, manage local accounts and groups, configure network settings, and automate entire workflows.

PowerShell embodies the shift from ad-hoc administration to structured, scalable system management. By uniting direct command execution with automation capabilities, it enables administrators to maintain systems more consistently and securely. Whether used for simple file manipulation, fine-grained access control, advanced network diagnostics, or fully automated maintenance routines, PowerShell demonstrates how a well-designed command-line environment can remain indispensable in modern IT practice.